

High-Performance Data Analysis

(Análise de Dados de Elevado Desempenho)

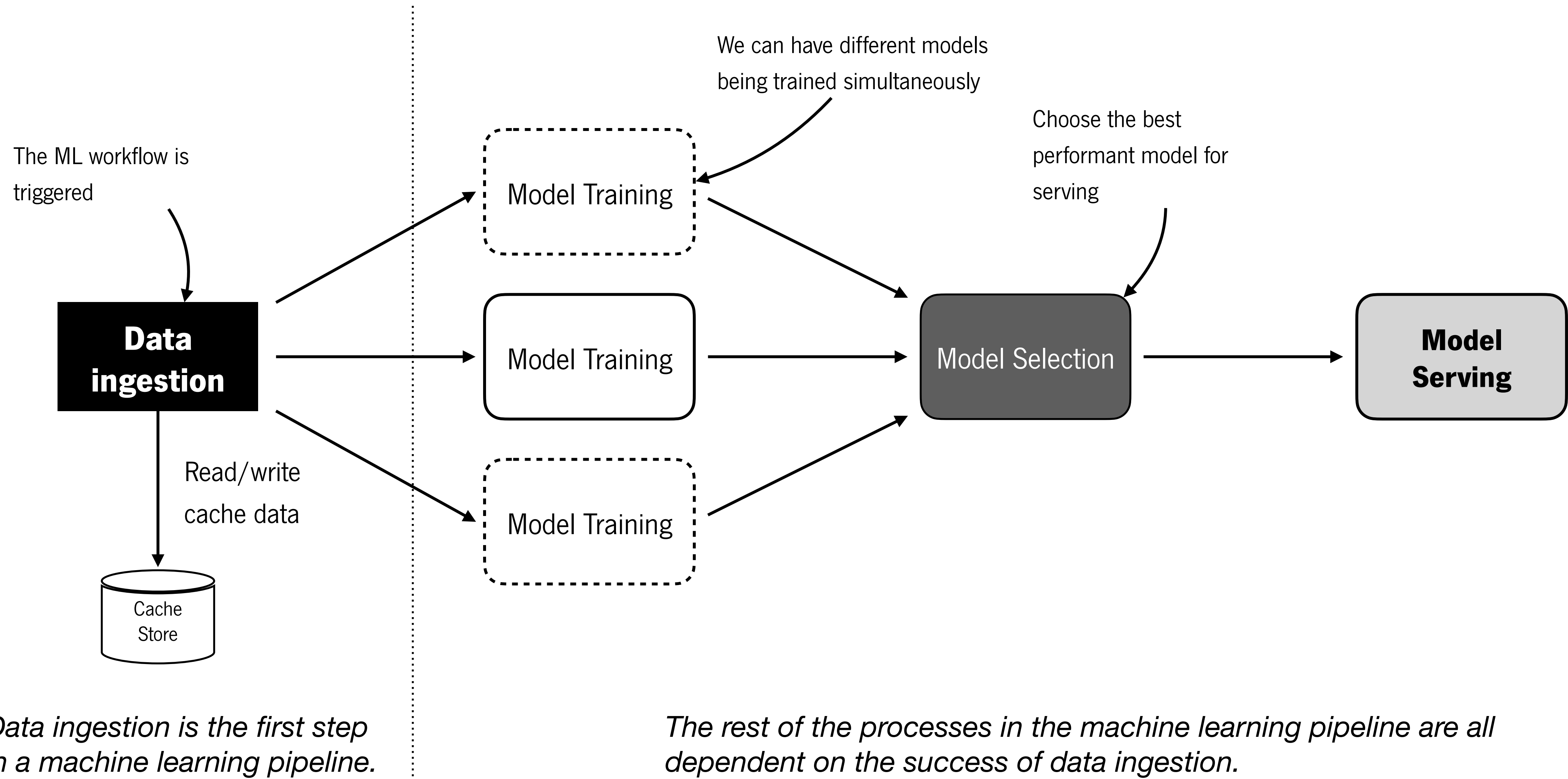
Shuffling, Batching, Sharding and Caching

University of Minho

2025-2026



Distributed Machine Learning



Distributed Machine Learning

- **Data ingestion** can be a **critical bottleneck**.
 - ▶ Bad ingestion → inefficient training → wasted resources!
 - ▶ Good ingestion = maximizing throughput + minimizing idle time
- Data ingestion patterns
 - ▶ Shuffling
 - ▶ Batching
 - ▶ Sharding
 - ▶ Caching

Distributed Machine Learning

● Data formats

- ▶ **CSV** - Easy to use; not optimized for large-scale training due to heavy parsing.
- ▶ **HDF5** - Hierarchical, efficient for large numerical datasets.
- ▶ **Parquet** - Columnar format, great for big data (used in Spark/Hive).
- ▶ **TFRecord** (Tensorflow specific) - Binary record storage.
- ▶ **JPEG** or **PNG** - For static storage of images.
- ▶ **Tensors** - Store as `.pt` or `.npy` for fast loading.

Note that, in PyTorch we typically use CSV, HDF5, or Parquet via custom *Dataset* classes.

Shuffling Pattern

The problem: Sequential Data Bias

- When training models on datasets where samples **are ordered** (e.g., sorted by class or timestamp), the model may learn patterns related to the order rather than the underlying data distribution.
- **Reduced model generalization to unseen data.**
- **Potential overfitting to specific sequences present in the training data.**

Shuffling Pattern

- **Shuffling** randomizes the order of data samples before each epoch. This practice prevents the model from learning unintended patterns based on data order, promoting better generalization.
- **Benefits:**
 - Enhances model's ability to generalize by exposing it to varied data sequences.
 - Prevents the model from relying on unintended order-based patterns.

```
from torch.utils.data import DataLoader, TensorDataset
import torch

# Sample dataset
X = torch.randn(1000, 10)
y = torch.randint(0, 2, (1000,))

dataset = TensorDataset(X, y)
dataloader = DataLoader(dataset, batch_size=32, shuffle=True, num_workers=4)

for batch_X, batch_y in dataloader:
    # Training step
    pass
```


Batching Pattern

The problem: Memory Constraints with Large Datasets

- Processing datasets with millions of samples individually is **computationally expensive** and **exceeds memory limits**.
- **Leading to:**
 - Increased training time.
 - Potential for out-of-memory errors.

Batching Pattern

- The solution is to **divide the dataset** into smaller groups (**batches**) and process each batch iteratively.
- **Benefits:**
 - Optimizes memory usage by loading only a subset of data at a time.
 - Enables efficient computation through vectorized operations.
 - Facilitates stable gradient estimates, improving convergence during training.

```
from torch.utils.data import DataLoader

train_loader = DataLoader(
    dataset, batch_size=64, shuffle=True, num_workers=4)

for batch in train_loader:
    inputs, labels = batch
    outputs = model(inputs)
```


Sharding Pattern

The problem: Inefficient Data Handling in Distributed Systems

- In a naive version, we assume that when training with large datasets on distributed systems:
 1. Each worker/node accesses the entire dataset, causing redundant I/O slowdowns.
 2. Inconsistent data access patterns can introduce biases and instability during training.

Sharding Pattern

- The process of breaking large datasets into smaller chunks spread across multiple machines is called sharding, and the smaller data chunks are called data shards.
- The dataset is divided into non-overlapping partitions; each worker reads only its shard.
- **Benefits:**
 - Parallelizes data loading.
 - Reduces network and storage pressure.
 - Ensures all samples are used **exactly once in each epoch** (important for correctness).
- **When to use:**
 - When training in a distributed manner (multiple GPUs, machines, or both).
 - When a dataset is very large and the I/O bandwidth is of utmost importance.

```
import torch
from torch.utils.data import DistributedSampler, DataLoader

sampler = DistributedSampler(dataset, num_replicas=4,
                             rank=rank)

train_loader = DataLoader(
    dataset, batch_size=64, sampler=sampler, num_workers=4)
```

Batching and Sharding

- ◎ **Batching helps process** large datasets and perform expensive operations under limited memory.
- ◎ **Sharding helps speed up** the training process as we add more worker machines that are responsible for model training on each data shard independently.
- ◎ Both batching and sharding patterns work well for the **model training process**; the dataset will be iterated thoroughly. However, epoch-based models require multiple scans of the dataset, performing batching and sharding continuously.

Caching Pattern

The problem: Repeated Data Loading Overhead

- In scenarios requiring multiple passes over the dataset (e.g., multiple epochs*), repeatedly loading data from disk incurs significant I/O overhead.
- **Consequences:**
 - Slower training due to frequent disk access.
 - Inefficient utilization of computational resources, as processors may idle while waiting for data.

*An **epoch** is a full pass-through of all the data we are training on, i.e., an epoch only ends when the model sees all the examples in the dataset.

Caching Pattern

- Store **frequently accessed data** in **faster storage backends** (e.g., RAM) to minimize disk I/O operations.
- **Benefits:**
 - Accelerates data retrieval, leading to faster training cycles.
 - Reduces dependency on slower disk operations.
 - Enhances overall system throughput by aligning data availability with processing speeds.

```
from torch.utils.data import Dataset, DataLoader
import torch

class CachedDataset(Dataset):
    def __init__(self, data):
        self.data = data
        self.cached_data = []

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        if not self.cached_data:
            self.cached_data = list(self.data)
        return self.cached_data[idx]

dataset = CachedDataset(data=torch.randn(1000, 10))

dataloader = DataLoader(dataset, batch_size=32,
                        shuffle=True, num_workers=4)

for batch_data in dataloader:
    # Training step
    pass
```

Interaction Between Patterns

- 1 Shuffle → Batch → Cache

- 2 Shuffle → Shard → Batch → Cache

- 3 Shuffle → Shard → Shuffle → Batch → Cache

Shuffle before batching, shard strategically, and cache when feasible.

Interaction Between Patterns

- ◎ **Shuffle Before Batching:** Always shuffle the data before creating batches to ensure each batch contains a representative mix of the dataset, promoting better model generalization.
- ◎ **Shard Strategically:** When distributing data across multiple workers, ensure that sharding is done thoughtfully to avoid data redundancy and imbalance, which can lead to biases and inefficiencies.
- ◎ **Cache When Feasible:** Utilize caching mechanisms when the dataset size permits, as it significantly speeds up data access times during training epochs.

PATTERN	PROBLEM	SOLUTION
BATCHING	SINGLE SAMPLES TOO SLOW	LOAD BATCHES
SHARDING	ONE WORKER CANNOT SCALE	SPLIT ACROSS WORKERS
CACHING	DISK TOO SLOW	STORE IN FAST MEMORY

Best Practices and Pitfalls

- **Use DataLoader for Batching and Shuffling:** The DataLoader class in PyTorch efficiently handles batching and shuffling, which are essential for effective training.
- **Enable Parallel Data Loading:** Set num_workers to a value greater than 0 to utilize multiple CPU cores for data loading, reducing the time the GPU waits for data.
- **Use Efficient Data Formats:** For large datasets, consider using binary formats like HDF5 or Parquet for structured data, which allow for faster reading compared to text formats like CSV.
- **Cache Data When Appropriate:** If the dataset is small enough to fit into memory, caching can significantly speed up data access by eliminating disk I/O.
- **Ensure Proper Shuffling:** Always shuffle your training data to prevent the model from learning unintended patterns based on data order.

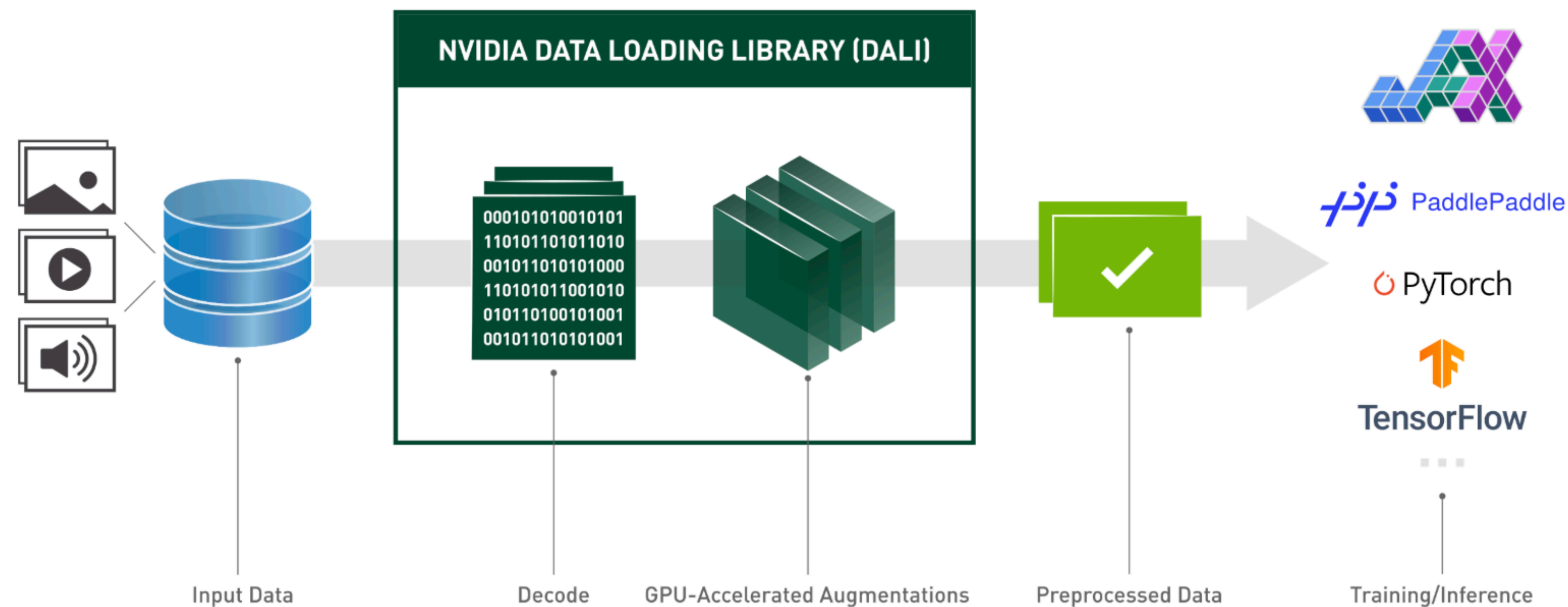
Best Practices and Pitfalls

Debugging possible bottlenecks of your model:

- `torch.utils.bottleneck` – a tool for debugging bottlenecks. It summarizes runs of your script with the Python profiler and PyTorch's autograd profiler.
 - <https://pytorch.org/docs/stable/bottleneck.html>
- `torch.profiler` - a tool that allows the collection of performance metrics during training and inference.
 - <https://pytorch.org/docs/stable/profiler.html>

Bonus

- You can use NVIDIA DALI for High-Performance Pipelines - <https://docs.nvidia.com/deeplearning/dali/user-guide/docs/index.html>



References

- Distributed Machine Learning Patterns. Yuan Tang. Manning Publications.
- <https://mljourney.com/efficient-data-loading-in-pytorch-tips-and-tricks-for-faster-training/>
- <https://www.restack.io/p/pytorch-answer-data-loading-best-practices-cat-ai>

Questions?